NEXUS — Module Log

Module 1: Core Foundation ✅

Files added

```
nexus/__init__.py
nexus/core/__init__.py
nexus/core/config.py
nexus/core/logging.py
nexus/core/di.py
nexus/core/interfaces.py
nexus/core/exceptions.py
tests/__init__.py
tests/test_config.py
tests/test_logging.py
tests/test_di.py
tests/test_interfaces.py
pyproject.toml
requirements.txt
.env.example
.gitignore
Dockerfile
README.md
```

Verified: `python -m unittest discover -s tests -v` → 32/32 passing, zero warnings. Also verified with an end-to-end smoke import (load config → configure logging → resolve from DI container) outside the test suite.

Not verified in this sandbox: `docker build` (Docker is not installed here) and installation of the `dev` extras (no network access in this sandbox). Both follow standard, reviewable patterns and should be validated in your local/CI environment before Module 2 lands on top of them.

Architecture

Module 1 establishes the layer every later module depends on, and nothing else:

- **`config.py`** — Immutable, validated configuration loaded from env vars / `.env`, with clear precedence (kwargs > real env > `.env` file > defaults) and secret redaction for safe logging. Built on `dataclasses` only — no pydantic

dependency
at this layer, so the foundation stays importable anywhere.

- **`logging.py`** — Structured JSON logging to a rotating file (for audit/ingestion) plus human-readable console output (for development), also stdlib-only.
- **`interfaces.py`** — `Protocol` definitions (`LLMProvider`, `SpeechToTextProvider`, `TextToSpeechProvider`, `MemoryStore`, `ActionExecutor`, `GuardrailEngine`, `Agent`) that every concrete implementation in later modules must satisfy. Using structural typing (Protocols) rather than abstract base classes means plugin authors can satisfy an interface without importing NEXUS's class hierarchy.
- **`di.py`** — A small, thread-safe DI container resolving providers by interface type, supporting both singleton and non-singleton (fresh-instance) registration.
- **`exceptions.py`** — One exception hierarchy rooted at `NexusError`, with

`GuardrailViolation` and `PermissionDeniedError` called out explicitly since those are the ones that must never be silently swallowed anywhere in the codebase.

Why this order: Every other module — the LangGraph agents, the voice pipeline, the ADB/Appium/Playwright executors, the guardrail engine itself — needs a place to read config from, a place to log to, a way to be swapped via DI, and a shared vocabulary of types (`ActionPlan`, `Message`, `ExecutionResult`, etc.). Building those first means Module 2 onward can be reviewed against a stable contract instead of a moving target.

Dependencies

Runtime: none (Python 3.11+ standard library only).
Dev/test: `pytest`, `pytest-asyncio`, `pytest-cov`, `mypy`, `ruff` (all optional — the test suite as shipped runs on `unittest` alone, zero installs required).

How to run

```
python -m unittest discover -s tests
-v
```

or, with dev tooling installed:

```
pip install -r requirements.txt
pytest -v
mypy nexus/
ruff check nexus/
```

Module 2: *(not started — awaiting your go-ahead)*

Proposed next module, in order of dependency: the **Security & Guardrail layer** (`nexus/security/`) — the deterministic policy engine (denylist, confirmation tiers, permission profiles, audit logging, rate limiting) implementing the `GuardrailEngine` Protocol from Module 1. I'd build this before any agent or executor, since every later module that touches

a real device or the filesystem needs to be built *against* a working guardrail from day one rather than have one bolted on afterward.

Alternative starting points if you'd rather go a different direction first: the **LLM Brain** (`nexus/agents/` + provider adapters for Claude/GPT/Gemini/Ollama), or the **Memory layer** (`nexus/memory/`). Let me know which you want, or confirm the guardrail layer and I'll proceed.